```
  end
  puts "ALERT: #{event[:description]}" if event[:condition].call
end
```

Both setup and event convert the block to a proc and store away the proc, in @setups and @events, respectively. These two top-level instance variables are shared by setup, event, and the main code.

The main code initializes @setups and @events, then it loads events.rb. The code in the events file calls back into setup and event, adding elements to @setups and @events.

With all the events and setups loaded, your program iterates through the events. For each event, it calls all the setup blocks, and then it calls the event.

You can almost hear the voice of Bill in your head, sounding a bit like Obi-Wan Kenobi: "Those top-level instance variables, @events and @setups, are like global variables in disguise. Why don't you get rid of them?"

Removing the "Global" Variables

To get rid of the global variables (and Bill's voice in your head), you can use a *Shared Scope* (*84*):

blocks/redflag_4/redflag.rb

```
lambda {
  setups = []
  events = []

  Kernel.send :define_method, :setup do |&block|
    setups << block
  end

  Kernel.send :define_method, :event do |description, &block|
    events << {:description => description, :condition => block}
  end

  Kernel.send :define_method, :each_setup do |&block|
    setups.each do |setup|
      block.call setup
    end
  end

  Kernel.send :define_method, :each_event do |&block|
    events.each do |event|
      block.call event
    end
  end
}.call
```